NANYANG TECHNOLOGICAL UNIVERSITY
School of Electrical and Electronic Engineering

NTU-TUM MSc (IC Design) Programme
Digital IC Design Course: NM6008

Laboratory Manual

Module 6

Layout of Matrix Multiplier Circuit

Course Instructors: A/Prof Lim Meng Hiot
A/Prof Gwee Bah Hwee
Dr Cheng Deruo
Sheng Shirui
Yu Guanhua
Lee Yujin
Liu Shukun

12nd Oct – 29st Oct 2026

Section 1: Introduction

This tutorial style laboratory manual document is meant for NTU-TUM Master of Science (MSc) course – Integrated Circuit (IC) Design, NM6008. This module introduces the concepts that serve as a bridge between modelling and synthesis to the actual physical layout level of an IC system. The following sub-sections state the objectives, the design tools/process used and the experimental design information of this module – Layout of Matrix Multiplier circuit.

1.1 Objective

Module 6 gives a quick introduction to the Place & Route layout implementation. The design example used is the Matrix Multiplier Circuit. The primary objective of this module is to let students familiarize themselves with the Cadence IC Place & Route layout tool (and the associated design methodology). The specific objectives include:

(i) To complete a layout using the standard library cells; and
(ii) To learn the design methodology of the semi-custom IC layout implementation.

1.2 Integrated Circuits (IC) Design Tools and Process Technology

Students will be exposed to the following IC design tools:

- Cadence Innovus (Cadence)
- Design Vision (Synopsys)

To learn more about these IC design tools, you may wish to read/refer to the following documents:

__, Cadence Innovus 20.1 User Guide, Cadence
__, Design Compiler™ User Guide. Synopsys.

The process technology used is the GF FDSOI 22nm CMOS fabrication process (different from previous technology process we used)

Section 2: Procedures

2.1 Working Environment

1. **Go** to your home directory and **choose** Module6. You will find the following folders:
   - **IO**

- **GDS**
- **RC**
- **Netlist**
- **Scripts**
- **LIB**
- **output_netlist**

Two other files will be available namely:

- **Syn_MatrixMultiplier_IO.globals**
- **Syn_MatrixMultiplier_IO.view**

In the **Netlist and IO** folder, files required to place and route are given. These files are usually provided by foundries or third party vendors depending on the process and standard cell libraries that you are using.

The **netlist** folder contains files below:

- **Syn_MatrixMultiplier.v**
- **Syn_MatrixMultiplier_IO.sdc**
- **Syn_MatrixMultiplier_IO.v**
- **Syn_MatrixMultiplier.io**

The file **Syn_MatrixMultiplier.v** is a Verilog file which is generated upon executing the synthesis of the Matrix Multiplier Circuit developed in Module 5. Alternatively, you can use your Matrix Multiplier Circuit if the input and output of your design are the same as those of the **Syn_MatrixMultiplier.v** here.

The file **Syn_MatrixMultiplier_IO.v** is a Verilog file in which the file **Syn_MatrixMultiplier.v** is instantiated. Input and output pads, input clock pad as well as power pads are added to the instantiated module.

[Note: The file **Syn_MatrixMultiplier_IO.v** is provided for ease of implementation for this lab exercise.]

The power pads are used to provide the power supply to the internal circuitry as well as to connect the internal circuitry to ground. The power pads are often treated as both input and output pads.

The file **Syn_MatrixMultiplier.io** provides IO pad locations for the design.

The **scripts** folder contains a TCL file named script.tcl which contains the sequential commands necessary to develop the complete layout of the matrix multiplier circuit. In the TCL file, the commands written without '#' signs before them indicate the actual valid commands. However the commands written with '#'

symbols are comments which are not executed. These comments serve as useful reminders or provide informative descriptions about the scripts.

The **timing_library** folder contains library timing files used for timing analysis, timing debugging, clock tree synthesis and timing aware routing.

2.2 Using Innovus

1. To open **Innovus**, type the following commands in a terminal:

   >> cd /home/"your home directory name"/Module6
   >> source cshrc_inno
   >> innovus

2. All the valid commands present in the TCL file should be executed in the terminal window sequentially and correspondingly the development of the circuit layout can be observed in the **Innovus** tool. The steps can be observed in Section 3.

Section 3: Steps in Implementing the Layout

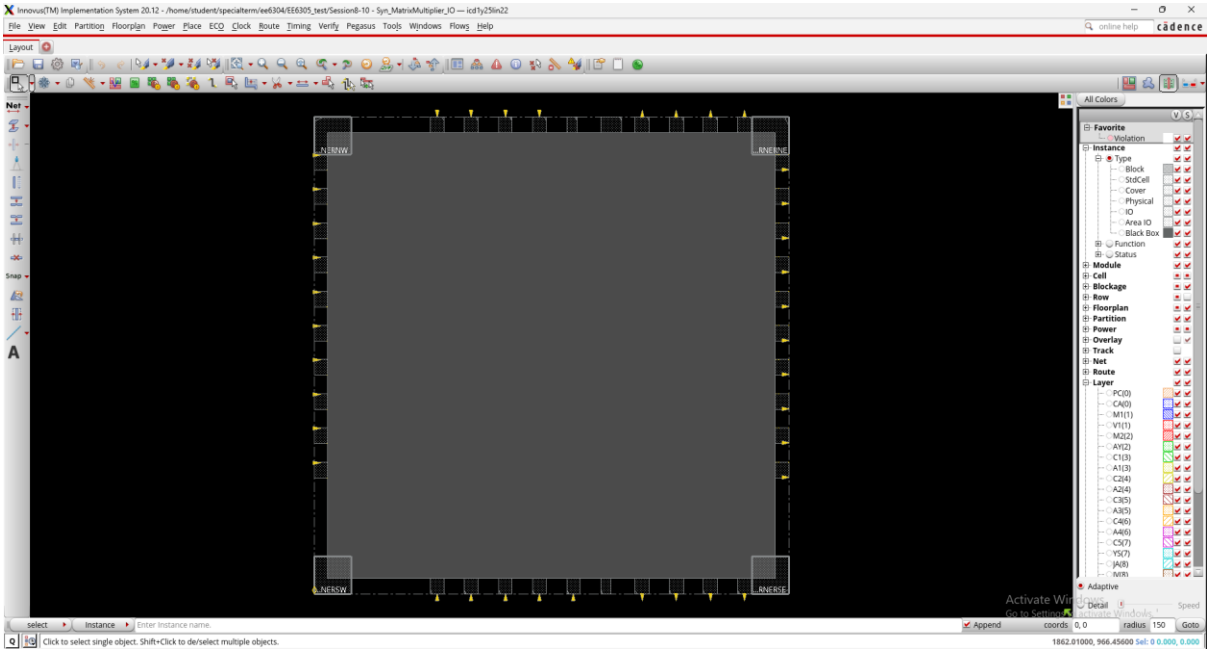
Note: All the commands need to be executed in the Innovus terminal.

1. Loading the global variables file

The first set of commands in the TCL file when executed will load the completed matrix multiplier Verilog code, including the port declarations onto the Innovus tool. The first set of commands is indicated below

```
>> source Syn_MatrixMultiplier_IO.globals
>> init_design
```

Upon execution, you will be able to visualize the core of the circuit present in the middle surrounded by the declared input and output ports as shown below.



Alternatively, you can load global variables file from GUI by clicking "**File**" and then choose "**Import Design…**". Click "**Load…**" in the "Design Import" window and choose `Syn_MatrixMultiplier_IO.globals` file.

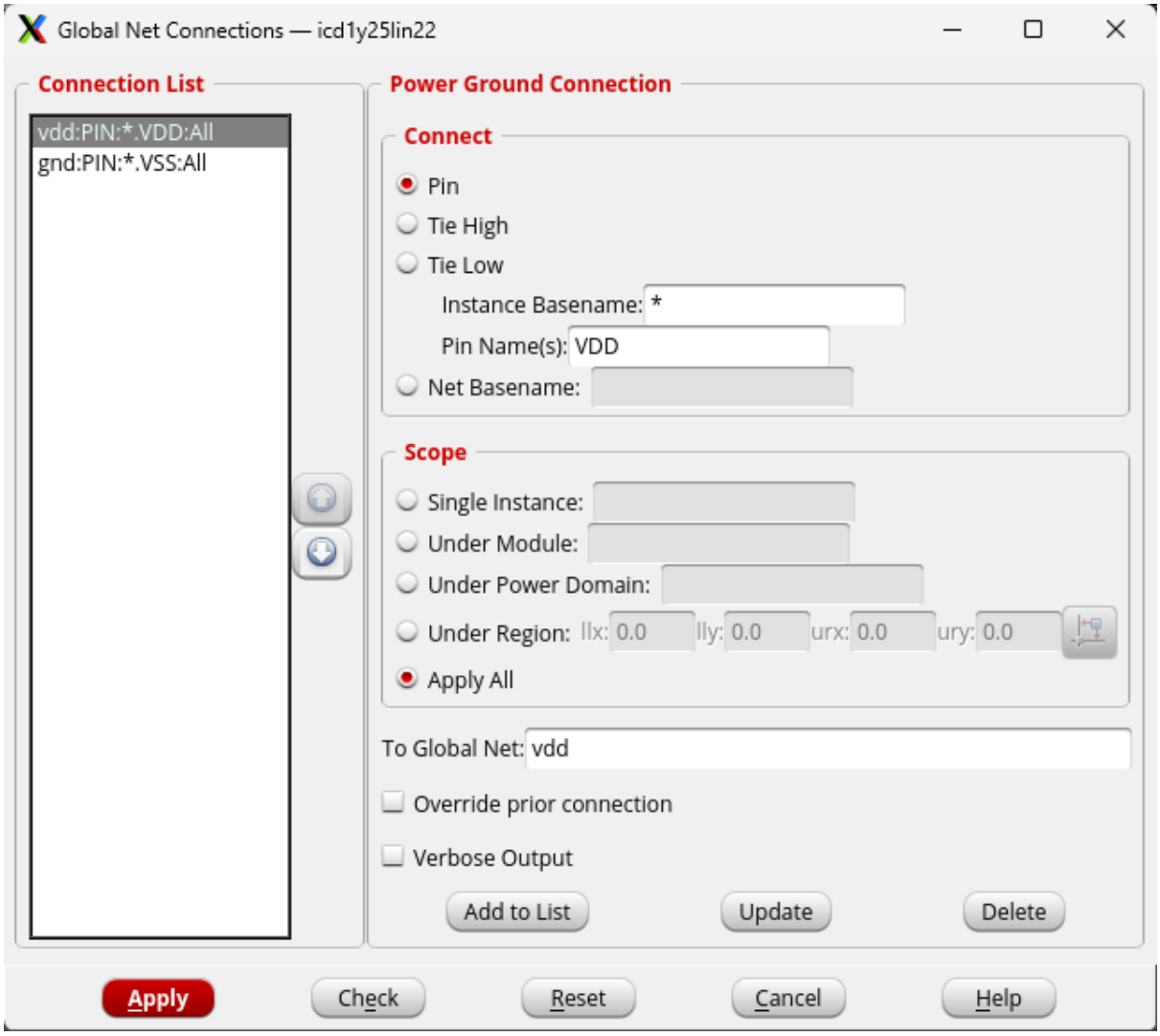
[Remarks: The `Syn_MatrixMultiplier_IO.globals` file links library information and design information. The *.globals* file provides a good reference if you need to setup other *.globals* file for other project.]

2. Global Net Connection

Global net connections specify which power/ground pin (of cells or pads) connects to which global net. The following commands are used to define the global power nets required in order to make global connections:

```
>> clearGlobalNets
>> globalNetConnect vdd -type pgpin -pin VDD -inst *
>> globalNetConnect gnd -type pgpin -pin VSS -inst *
```

To specify global nets from GUI, click on "**Power**" then choose "**Connect Global Nets**…" to open "Global Net Connection" window to add global power nets information. Click "**Apply**" after all information has been specified.



3. Floorplanning

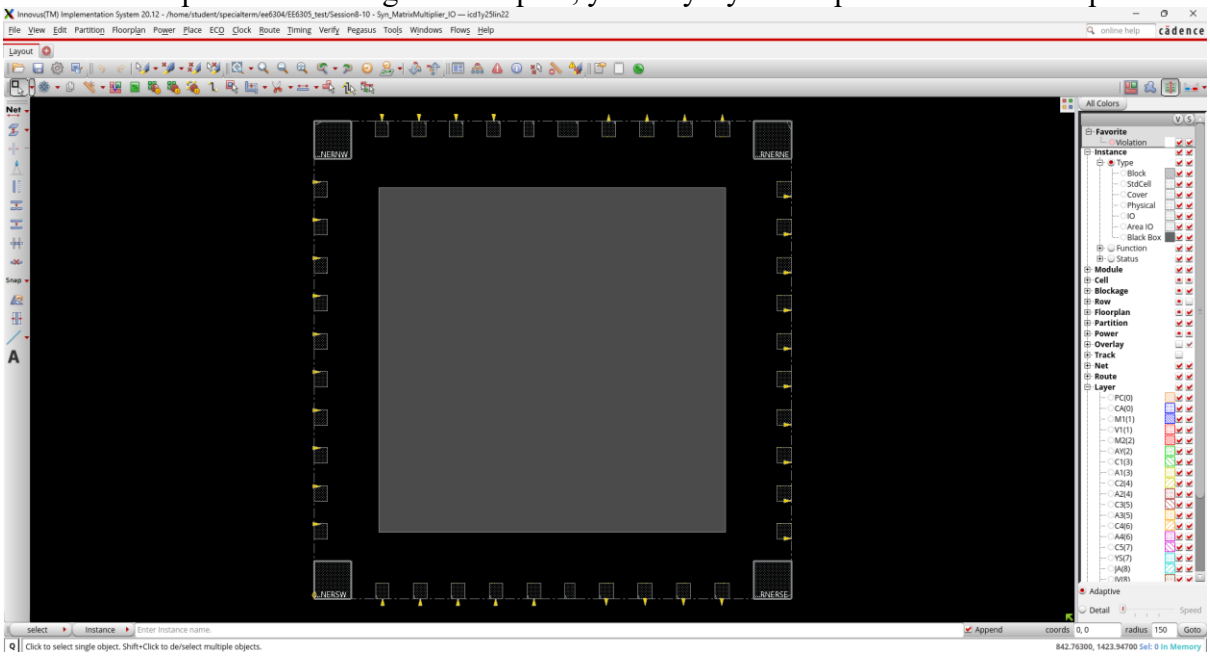
We now need to generate the floorplan for the matrix multiplier circuit. It would be desirable to generate the floorplan by using GUI. From GUI, first click on "**Floorplan**" and then choose "**Specify Floorplan**" to open "Specify Floorplan" window. Provide floorplan information as shown in the picture and click "Apply".



The floorplan can also be modified by using commands below:

```
>> getIoFlowFlag
>> setIoFlowFlag 0
>> floorPlan -site GF22_SST -d 1400 1400 150 150 150 150 -noSnapToGrid
```

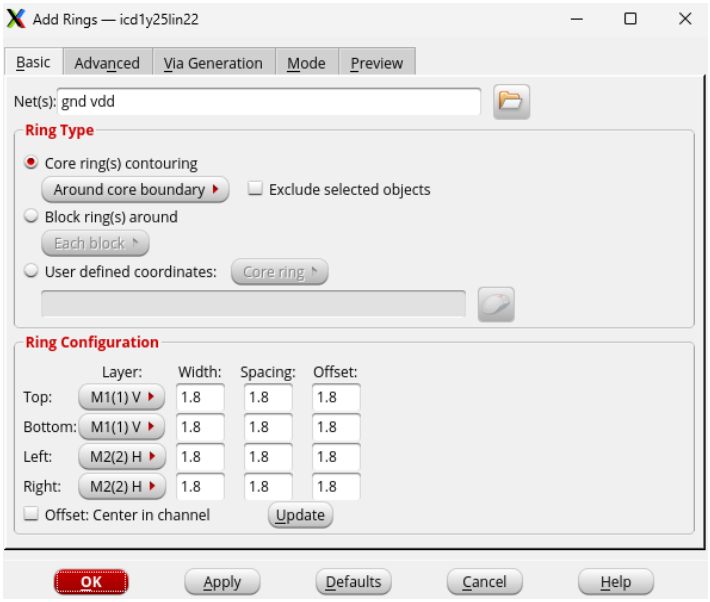
The floorplan is modified based on specification you have provided. Although above-mentioned steps will create a good floorplan, you may try and explore different floorplans.



4. Power Rings Creation

The next step involved in developing the layout is to create a connection between the input/output pads and the internal core circuitry so that the external power supply can be made

available to the internal circuit. TO add power rings from GUI, click on "**Power**", then click "**Power Planning**" and choose "**Add Ring…**" to open up "Add Rings" window. Add the power nets that you would like to connect and specifications for rings before clicking "**OK**" or "**Apply**".

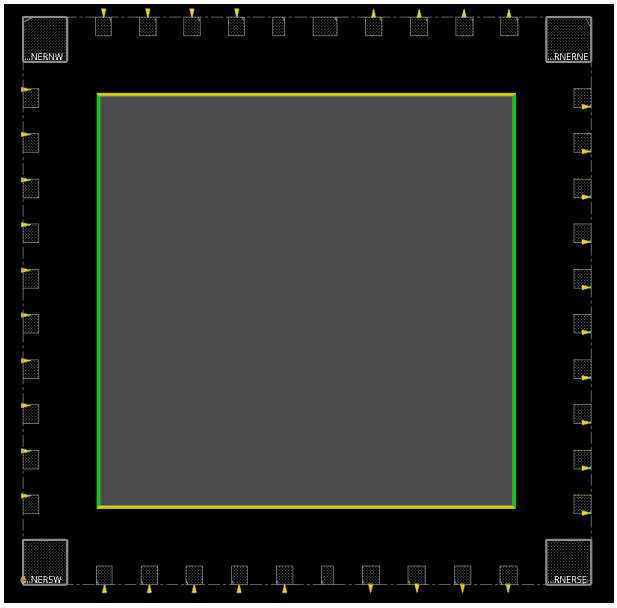


Power rings can also be added by execution of the following commands.

```
>> setAddRingMode -ring_target default -extend_over_row 0 -ignore_rows 0 -avoid_short 0 -skip_crossing_trunks none -stacked_via_top_layer LB -stacked_via_bottom_layer M1 -via_using_exact_crossover_size 1 -orthogonal_only true -skip_via_on_pin { standardcell } -skip_via_on_wire_shape { noshape }

>> addRing -nets {gnd vdd} -type core_rings -follow core -layer {top C2 bottom C2 left C1 right C1} -width {top 1.8 bottom 1.8 left 1.8 right 1.8} -spacing {top 1.8 bottom 1.8 left 1.8 right 1.8} -offset {top 1.8 bottom 1.8 left 1.8 right 1.8} -center 0 -threshold 0 -jog_distance 0 -snap_wire_center_to_grid None
```

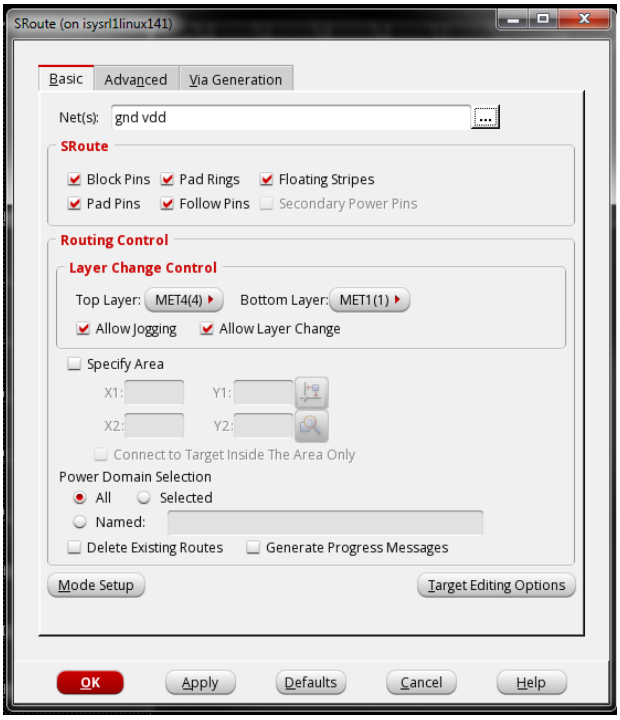
The Innovus window displays the result as shown below:



5. Special Nets Routing

Special nets routing (SRoute) creates power/ground stripes, connects core/block rings to the power/ground pads, and power/ground nets for pads.

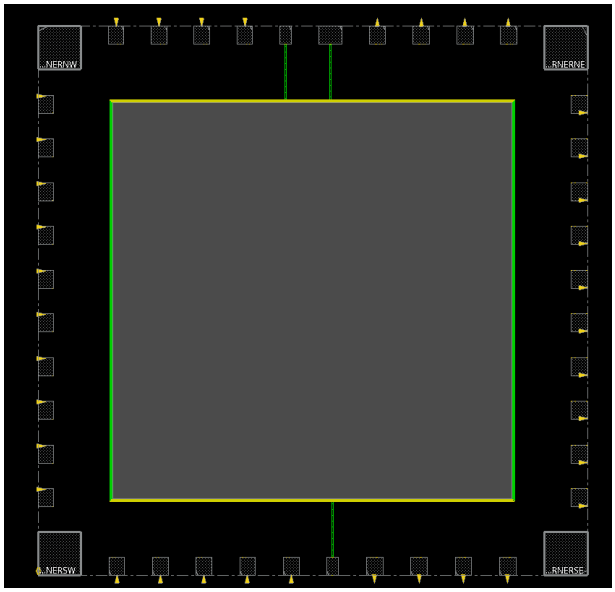
From GUI, click on "**Route**" and choose "**Special Route…**" to open "**Special Route**" window. Fill in the names of special nets and click "**OK**" or "**Apply**".



Special routing can also be achieved with the following commands:

```
>> setSrouteMode -viaConnectToShape { noshape }
>>  sroute  -connect  {  blockPin  padPin  padRing  corePin
floatingStripe  }  -layerChangeRange  {  MET1(1)  MET4(4)  }  -
blockPinTarget { nearestTarget } -padPinPortConnect { allPort
oneGeom } -padPinTarget { nearestTarget } -corePinTarget {
firstAfterRowEnd } -floatingStripeTarget { blockring padring
ring stripe ringpin blockpin followpin } -allowJogging 1 -
crossoverViaLayerRange { MET1(1) MET4(4) } -nets { gnd vdd } -
allowLayerChange 1 -blockPin useLef -targetViaLayerRange {
MET1(1) MET4(4) }
>> setLayerPreference node_layer -isVisible 0
>> setLayerPreference C1 -isVisible 1
>> setLayerPreference A1 -isVisible 1
>> setLayerPreference C2 -isVisible 1
>> selectWire 628.9500 1354.0000 633.0500 1354.1180 3 gnd
>> uiSetTool stretchWire
>>  editResize  -direction  y  -offset  -148.259  -side  low  -
keep_center_line auto
>> deselectAll
>> selectWire 742.9500 1354.0000 747.0500 1354.1180 3 vdd
>>  editResize  -direction  y  -offset  -148.384  -side  low  -
keep_center_line auto
>> editTrim
>> deselectAll
>> selectWire 748.9500 45.8820 753.0500 46.0000 3 gnd
>>  editResize  -direction  y  -offset  148.524  -side  high  -
keep_center_line auto
>> editTrim
>> deselectAll
```

The Innovus window generated is as shown below:



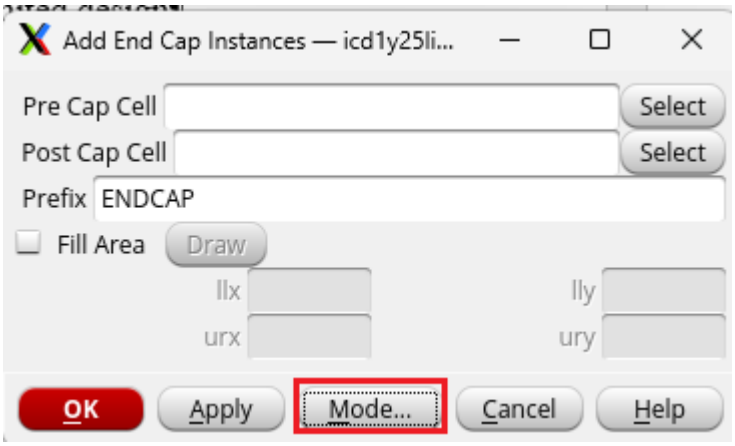
6. IO pad Filler Placement

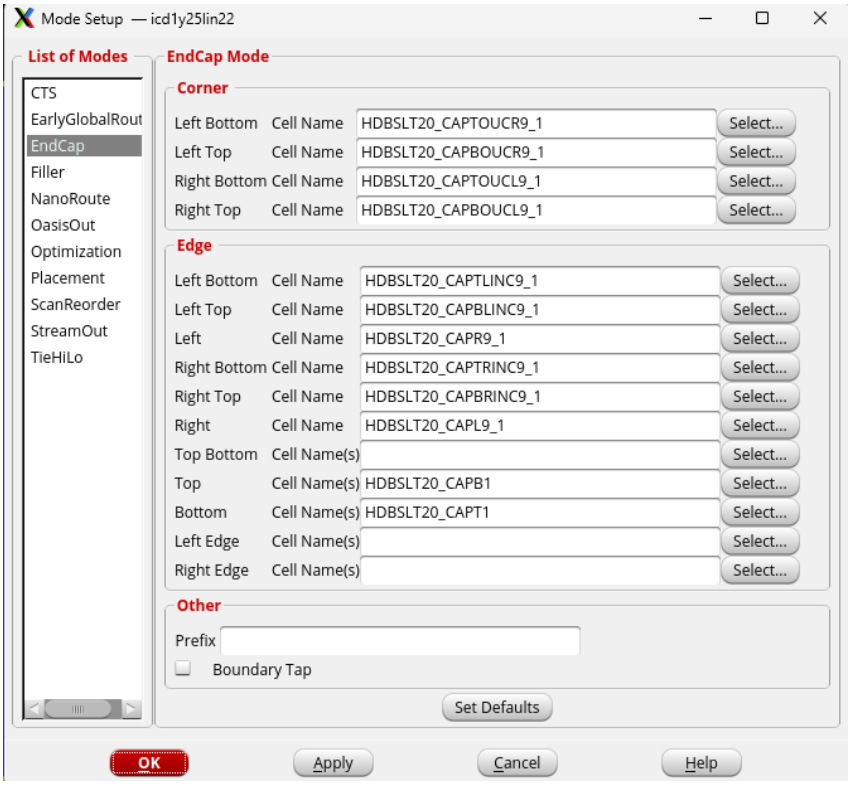
- **There should be no spacing between pads**
  - Therefore adding pad filler is necessary for core limited design
- PAD filler must be added from wider fillers to narrower ones, or there may have some DRC/LVS violations after PAD filler insertion

```
>> addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_V
     IN22FDX_GPIO18_10M19S40PI_FILL10_V
     IN22FDX_GPIO18_10M19S40PI_FILL5_V
     IN22FDX_GPIO18_10M19S40PI_FILL1_V" -prefix IOFILLER_N -
     side n
>> addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_V
     IN22FDX_GPIO18_10M19S40PI_FILL10_V
     IN22FDX_GPIO18_10M19S40PI_FILL5_V
     IN22FDX_GPIO18_10M19S40PI_FILL1_V" -prefix IOFILLER_S -
     side s
>> addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_H
     IN22FDX_GPIO18_10M19S40PI_FILL10_H
     IN22FDX_GPIO18_10M19S40PI_FILL5_H
     IN22FDX_GPIO18_10M19S40PI_FILL1_H" -prefix IOFILLER_W -
     side w
>> addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_H
     IN22FDX_GPIO18_10M19S40PI_FILL10_H
     IN22FDX_GPIO18_10M19S40PI_FILL5_H
     IN22FDX_GPIO18_10M19S40PI_FILL1_H" -prefix IOFILLER_E -
     side e
```

7. End cap Placement (subject to the requirement of the process/library)

End cap cells are needed to fulfil the well tie off specification and well extension requirement for DRC. To add them from GUI, click on "**Place**", select "**Physical Cell**" and choose "**Add End Cap…**". Choose appropriate End caps in "Mode" for placement and click "**OK**" or "**Apply**".



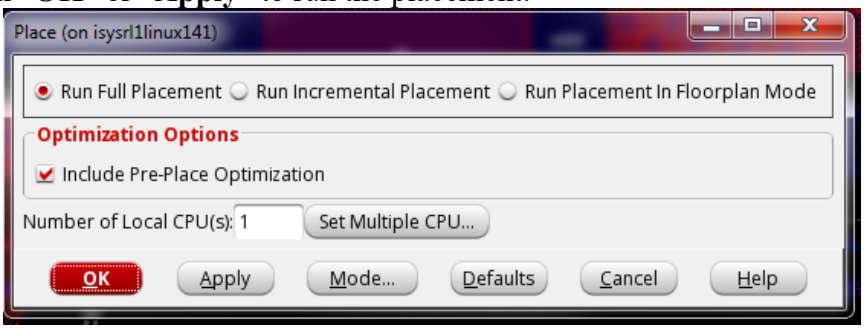


End cap cells can also be added by using the command below:

```
>> snapFPlan -all
>> set CELL_PREFIX "HDBSLT20"
>> setEndCapMode -reset
>>
>>
>> setEndCapMode -reset
>>  setEndCapMode \
    -topEdge "${CELL_PREFIX}_CAPB1" \
    -bottomEdge "${CELL_PREFIX}_CAPT1" \
    -leftEdge "${CELL_PREFIX}_CAPR9_1" \
    -rightEdge "${CELL_PREFIX}_CAPL9_1" \
    -rightTopCorner "${CELL_PREFIX}_CAPBOUCL9_1     " \
    -rightBottomCorner "${CELL_PREFIX}_CAPTOUCL9_1 " \
    -leftTopCorner "${CELL_PREFIX}_CAPBOUCR9_1      " \
    -leftBottomCorner "${CELL_PREFIX}_CAPTOUCR9_1   " \
    -leftBottomEdge "${CELL_PREFIX}_CAPTLINC9_1" \
    -lefttopEdge "${CELL_PREFIX}_CAPBLINC9_1" \
    -rightBottomEdge "${CELL_PREFIX}_CAPTRINC9_1" \
    -righttopEdge "${CELL_PREFIX}_CAPBRINC9_1" \
    -fitGap true
>> addEndCap -prefix ENDCAP
```

8. Standard Cell Placement

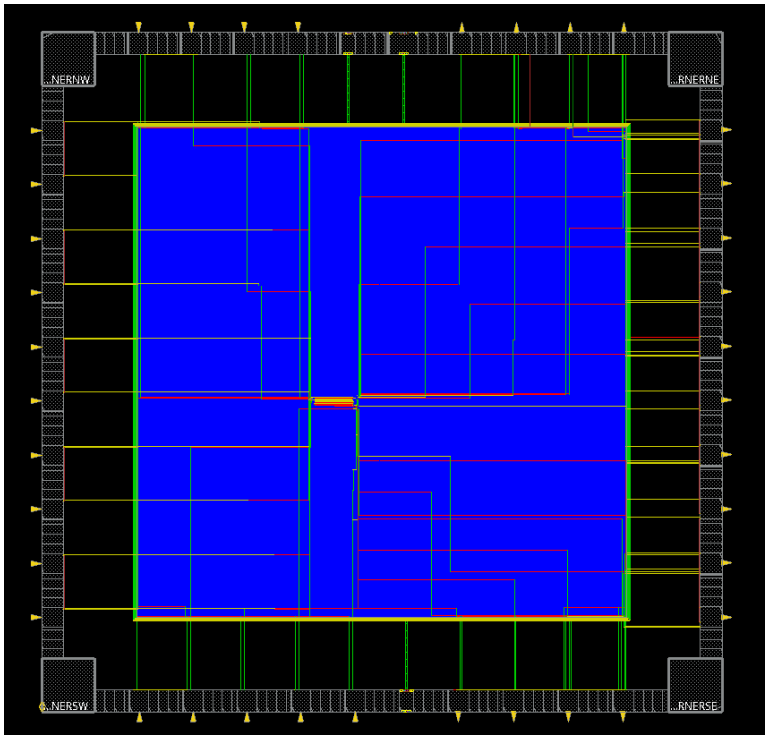
The following step involves placement of standard library cells used in the circuit. To place the cells from GUI, click on "**Place**" and select "**Place Standard Cell…**". On the "Place" window, click "**OK**" or "**Apply**" to run the placement.



The text commands for standard cell placement are:

```
>> setPlaceMode -fp false
>> placeDesign
```

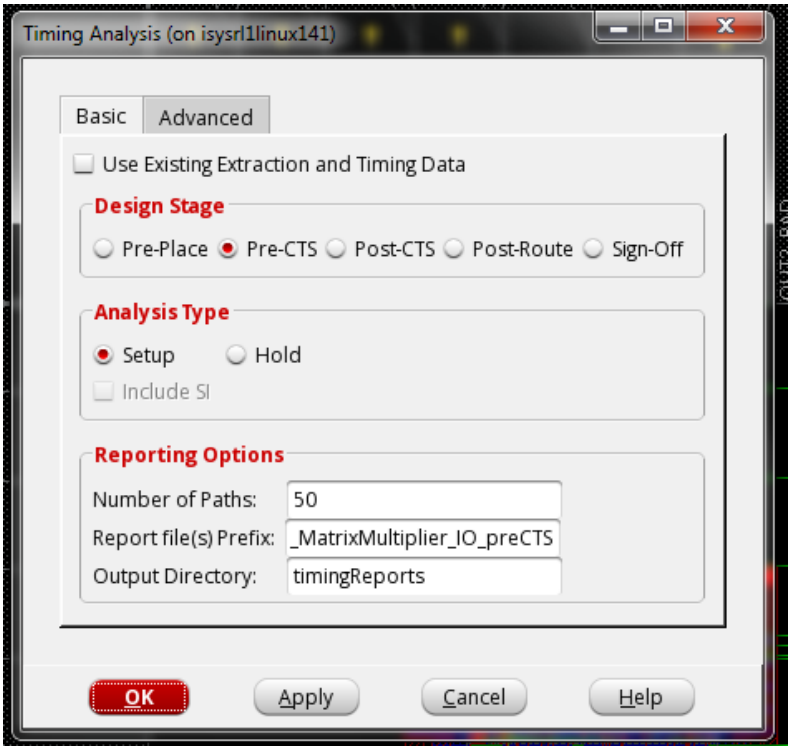
After running placement, the standard cells are placed in core region and design has been trial routed. The result as shown below:



9. Clock Tree Synthesis

In this part, Clock Tree Synthesis (CTS) and optimization will be done.

Before running CTS, timing analysis should be done to check if there is any setup time violation exists in any of the signal paths. To perform the check from GUI, click on "**Timing**", then select "**Report Timing…**". Check "**Pre-CTS**" under design stage and "**Setup**" Analysis Type and click "**OK**" or "**Apply**".



You will observe that a folder named "**timingReports"** will be generated within the folder Module 6. Select the timing Reports folder and choose the file named **Syn_MatrixMultiplier_IO_preCTS.summary**. You may visualise the contents of the file as similar to what is shown in the following window:

```
------------------------------------------------------------------
     optDesign Final Summary
------------------------------------------------------------------

Setup views included:
 typical rcworst_DP

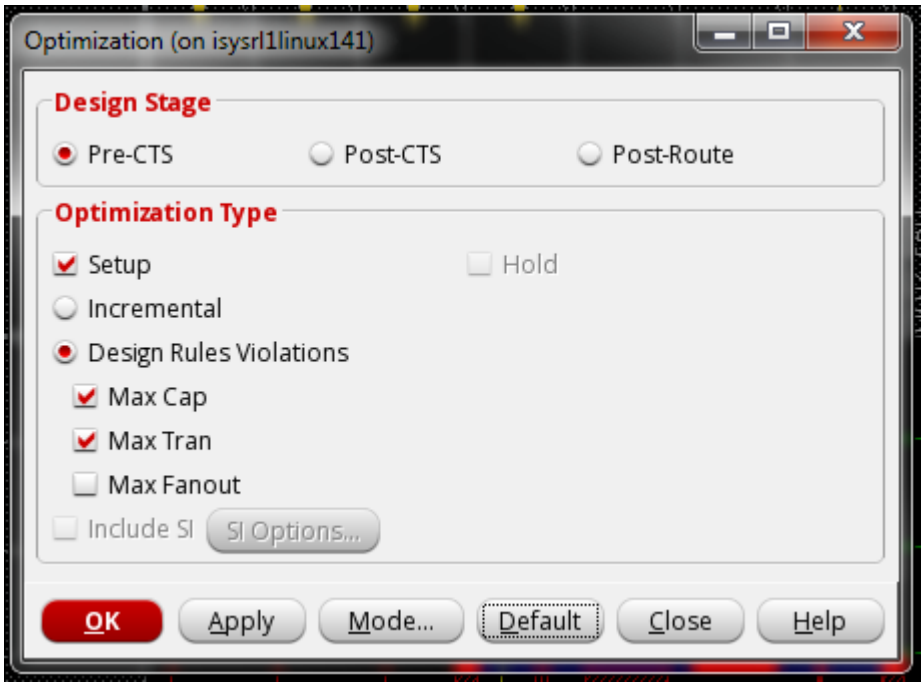
+--------------------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):| 90.034  | 96.646  | 90.034  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |
|          All Paths:|   126   |   53    |   90    |
+--------------------+---------+---------+---------+

+----------------+-------------------------------+------------------+
|                |              Real             |       Total      |
|    DRVs        +------------------+------------+------------------|
|                |  Nr nets(terms)  | Worst Vio  |  Nr nets(terms)  |
+----------------+------------------+------------+------------------+
|   max_cap      |      0 (0)       |   0.000    |      0 (0)       |
|   max_tran     |      0 (0)       |   0.000    |      0 (0)       |
|   max_fanout   |      0 (0)       |     0      |      0 (0)       |
|   max_length   |      0 (0)       |     0      |      0 (0)       |
+----------------+------------------+------------+------------------+

Density: 0.023%
Routing Overflow: 0.00% H and 0.00% V
```

You will be able to see the table containing setup mode parameters. Violating paths is one of those parameters and in the above case no violating path exists as the number of violating paths is zero in all the mentioned cases. But if any setup time violating path exists, it should be optimized.

To optimise design from GUI, click on "**ECO**" and select "**Optimize Design…**". In "Optimization" window, select "**Pre-CTS**" under Design Stage and choose "**Setup**" under Optimization Type. If there are any design rules violations, choose "**Design Rules Violations**" and select appropriate types. If not, choose "**Incremental**".



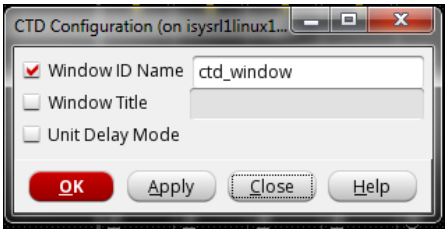
You can perform the "Hold" time analysis same way as "Setup" time analysis. However, note that there is no "Hold" time optimization for "Pre-CTS". It can only be optimized Post-CTS. Pre-CTS analysis and optimization can also be performed using the commands below:

```
>> timeDesign -preCTS -idealClock -pathReports -drvReports -
slackReports -numPaths 50 -prefix
Syn_MatrixMultiplier_IO_preCTS -outDir timingReports
>> setOptMode -fixCap true -fixTran true -fixFanoutLoad false
>> optDesign -preCTS
```

CTS can be performed after pre-CTS analysis and optimization. CTS can only be performed from text command and the commands are:

```
>> create_ccopt_clock_tree_spec
>> ccopt_design -prefix cts
```

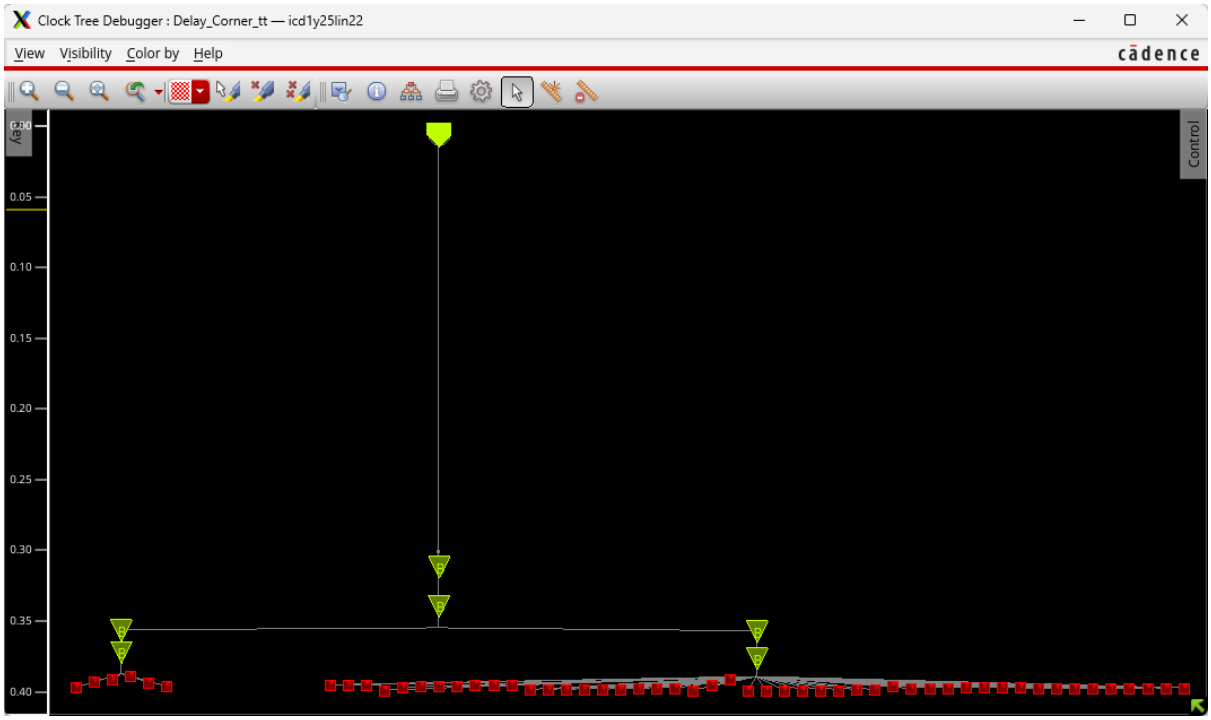
The clock tree can be view and trace from "Clock Tree Debugger" window. To open the window, click on "**Clock**" then select "**CCOpt Clock Tree Debugger**". By clicking "**OK**" on "CTD configuration" window, you will see the "Clock Tree Debugger" window.



Alternatively you can use the command below to open the "**Clock Tree Debugger**" window directly:

```
>> ctd_win -id ctd_window
```

The "**Clock Tree Debugger**" window is shown below. The information on clock tree such as clock root, buffer, buffer specification, connections and locations can be view, and clock tree can be trace from the window.



In a similar way, the setup time and hold time violation of signal paths after clock synthesis can be checked. The procedure is very similar to the check performed before clock tree synthesis.

The steps involved in checking and optimizing the setup time violation is similar to the steps mentioned above except instead of **pre-CTS** you have to choose the option **post-CTS** wherever required. Similarly the file which shows the violating paths information after clock synthesis is a file named **Syn_MatrixMultiplier_IO_postCTS.summary** generated within the timing Reports folder.

The steps involved in checking and optimizing the hold time violation is similar to the steps mentioned above except instead of **pre-CTS,** you have to choose the option **post-CTS** wherever required. Similarly the file which shows the violating paths information after clock synthesis is a file named **Syn_MatrixMultiplier_IO_postCTS_hold.summary** generated within the timing Reports folder.

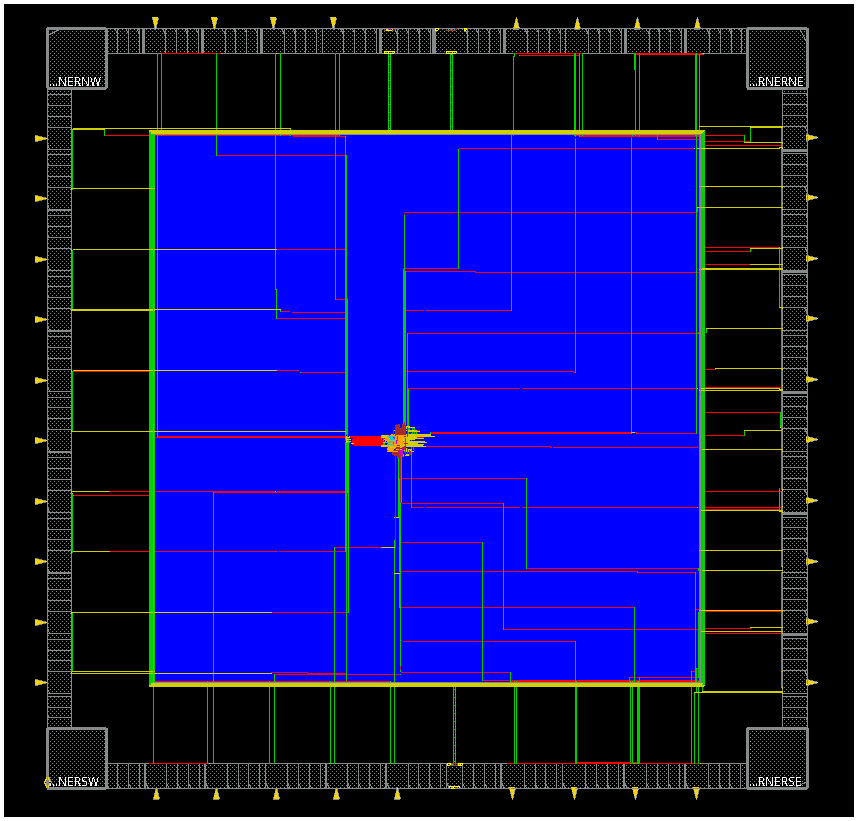
10. NanoRouting

The next major task required in the layout development is the formation of interconnections both within and between the different functional blocks present in the core circuit in order to realize the circuit functionality.

This task of routing also be performed by the following command:

```
>> source  ./scripts/Route.tcl
```

The Innovus window clearly displays the interconnection formed as given below:



Similar to pre-clock synthesis and post-clock synthesis, the setup time and hold time violations can be checked and optimized after the completion of routing. The analysis setting should be changed to on-chip variation to do post-Route analysis. To change from GUI, click on "**Tools**", select "**Set Mode**" and choose "**Specify Analysis Mode…**". On "Specify Analysis Mode…" window, choose "**On-Chip Variation**" under "**Timing Mode**".

It can also be done using text command as below:

```
>> setExtractRCMode -engine postRoute -effortLevel signoff -coupled true -capFilterMode relAndCoup -coupling_c_th 0.1 -total_c_th 0 -relative_c_th 1 -lefTechFileMap 22FDSOI_10M_2Mx_5Cx_1Jx_2Qx_LB_qrc_leftech.map
>> setAnalysisMode -cppr none -clockGatingCheck true -timeBorrowing true -useOutputPinCap true -sequentialConstProp false -timingSelfLoopsNoSkew false -enableMultipleDriveNet true -clkSrcPath true -warn true -usefulSkew true -analysisType onChipVariation -log true

>> redirect -quiet {set honorDomain [getAnalysisMode -honorClockDomains]} > /dev/null
>> timeDesign -signoff -pathReports -drvReports -slackReports -numPaths 50 -prefix CHIP_TOP_signOff -outDir timingReports
>> redirect -quiet {set honorDomain [getAnalysisMode -honorClockDomains]} > /dev/null
```

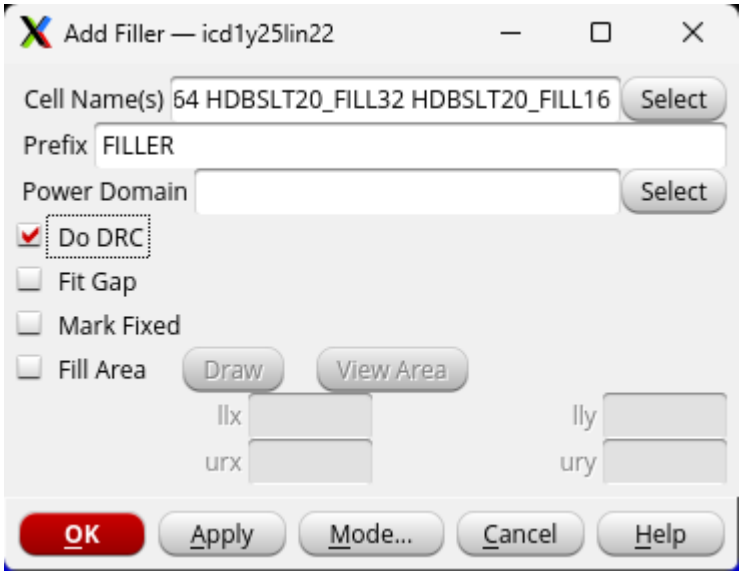
```
>> timeDesign -signoff -hold -pathReports -slackReports -
numPaths 50 -prefix CHIP_TOP_signOff -outDir timingReports
```

The steps involved in checking and optimizing the setup time violation is similar to the steps mentioned above except instead of **pre- CTS** or **post-CTS** you have to choose the option **post-Route** wherever required. Similarly the file which shows the violating paths information after routing is a file named **Syn_MatrixMultiplier_IO_postRoute.summary** generated within the timing Reports folder.

The steps involved in checking and optimizing the hold time violation is similar to the steps mentioned above except instead of **pre-CTS** or **post-CTS**, you have to choose the option **post-Route** wherever required. Similarly the file which shows the violating paths information after routing is a file named **Syn_MatrixMultiplier_IO_postRoute_hold.summary** generated within the timing Reports folder.

11. Filler Cells Placement

The next step involved in completing the layout is the addition of core filler cells. The filler cells are required for continuity of wells and to avoid design rule violation. To add them from GUI, click on "**Place**", select "**Physical Call**" then choose "**Add Filler…**". Specify filler cells names and check "**Do DRC**" on "**Add Filler**" window.

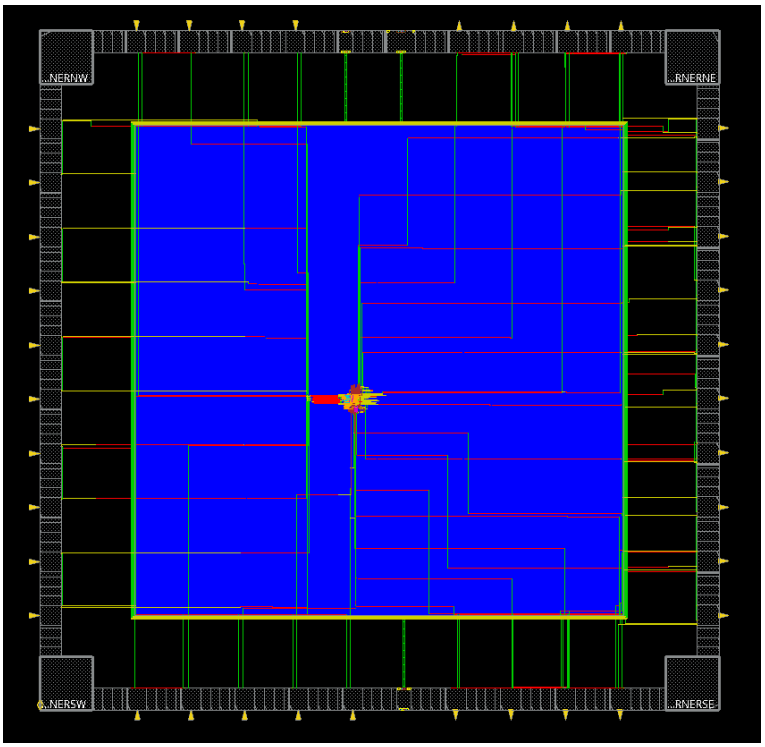


This can also be achieved by the following command:

```
>> getFillerMode -quiet
>>    addFiller    -cell    HDBSLT20_FILL64    HDBSLT20_FILL32
HDBSLT20_FILL16  HDBSLT20_FILL8  HDBSLT20_FILL4  HDBSLT20_FILL3
HDBSLT20_FILL2 HDBSLT20_FILL1 -prefix FILLER -doDRC
```

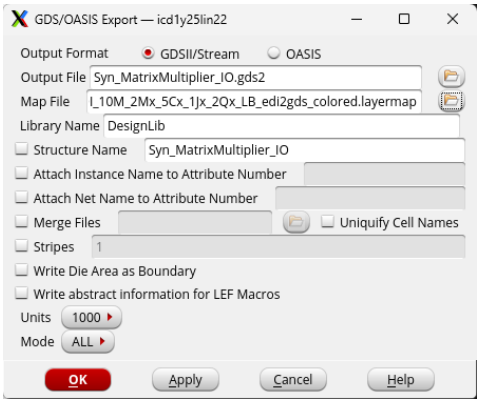
```
>> source  ./scripts/Route.tcl
```

The Innovus window is thus modified as shown (note that wires are hidden):



12. GDS2 Extraction

Thus the complete layout of the matrix multiplier circuit is successfully placed and routed. To export out the layout (in GDS format) from GUI, click on "**File**", select "**Save**" and choose "**GDS/OASIS…**". On "GDS/OASIS Export" window, specify the output file name and Map file (provided by foundry), and click "**OK**".



You can also extract the layout in GDS format using command as follows:

```
>> streamOut Syn_MatrixMultiplier_IO.gds2 -mapFile
GDS/22FDSOI_10M_2Mx_5Cx_1Jx_2Qx_LB_edi2gds_colored.layermap -
libName DesignLib -structureName Syn_MatrixMultiplier_IO -
units 1000 -mode ALL
```

13. Parasitic Extraction

The parasitic information from the layout can be extracted for post-layout simulation and verification. To extract from GUI, click "**Timing**", and select "**Extract RC…**". On "Extract RC" window, check the parasitic file format that you need (in this case .spef) and click "**OK**".

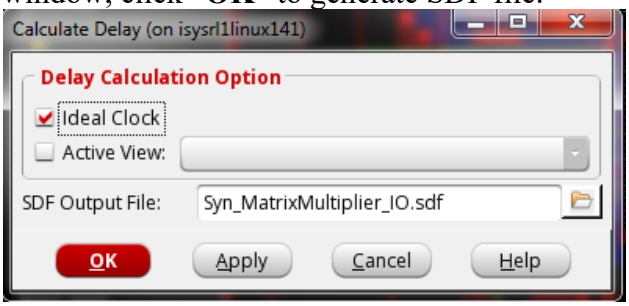


The parasitic extraction can be done by the following commands:

```
>> rcOut -spef Syn_MatrixMultiplier_IO.spef -rc_corner typical
```

14. Timing File Generation

The timing file/ SDF (Standard Delay Format) file can also be generated for post-layout simulation and verification. To generate from GUI, click "**Timing**" and select "**Write SDF…**". On "Calculate Delay" window, click "**OK**" to generate SDF file.

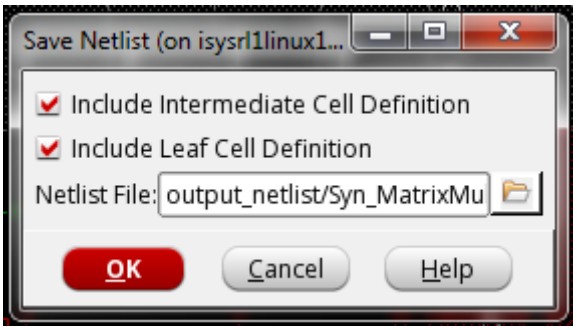


It can also be done using the command as follows:

```
>> all_hold_analysis_views
>> all_setup_analysis_views
>> write_sdf Syn_MatrixMultiplier_IO.sdf
```

15. Post-Layout Netlist Extraction

The routed Verilog netlist is required to do post-layout verification and simulation. To generate it from GUI, click on "**File**", select "**Save**" and choose "**Netlist…**". On "Save Netlist" window, click "**OK**" to save the Verilog netlist.



The Verilog file can also be extracted using the command below:

```
>> saveNetlist output_netlist/Syn_MatrixMultiplier_IO.v
```